table-segment.py in Plain Words

Oct 19, 2026 · @Pranay

How to read this

This document explains what table-segment.py does in everyday words, with no numbers and no formulas. Each piece gets three short answers: **what it does**, **why it is needed**, and **how it solves the problem**.

This is the top level. Once it makes sense, we go one level deeper at a time, and only then do details come in.

Throughout, "English" means the original document and "Spanish" means its translation. The script works the same way for any pair of languages.

The problem

The script finds, for every cell of every table in the English PDF, the matching cell in the Spanish PDF.

Paragraphs were already handled well by the existing pipeline. Tables were not. The old way assumed the tables line up neatly: the first table matches the first table on the same page, and each cell sits in the same row and column on both sides.

That assumption breaks because Spanish text is longer than English:

- The Spanish PDF has more pages, so page breaks cut tables at different places.
- One English table can come out as two separate Spanish tables.
- Some tables exist in one language only.
- Azure sometimes reads one row as two rows.

Once one thing shifts, everything after it is matched wrongly or not at all. That is the failure this script fixes.

The one big idea

A translator changes the words, but never changes the order of a table.

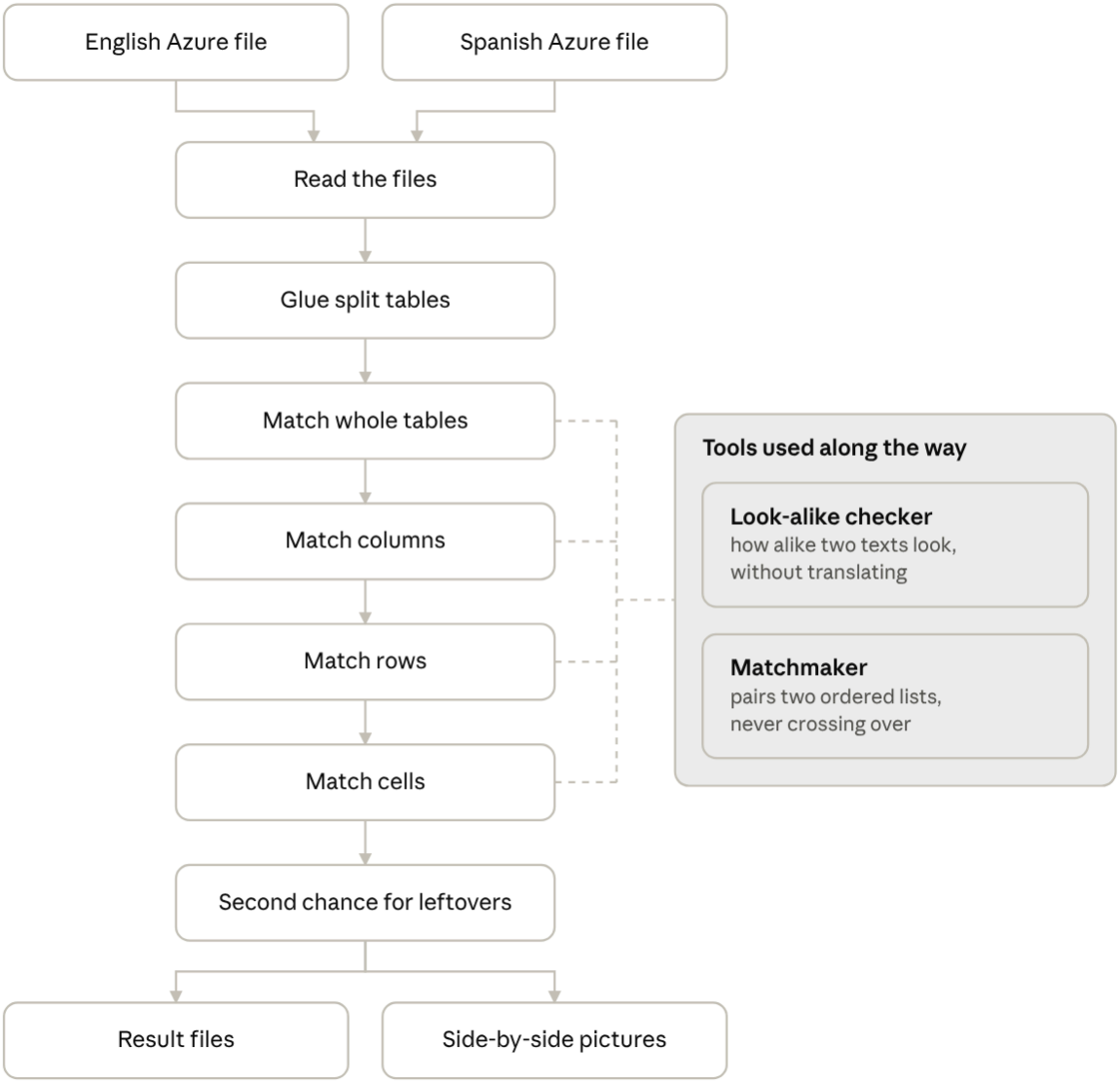
The first row in English is still the first row in Spanish. The price column is still the price column. So the script never needs to understand Spanish. It walks through both tables side by side, in order, and lines them up.

A few things look the same in both languages: money amounts, percentages, and words that are spelled alike. These act like pins that hold the two sides in place. Everything else falls into place between the pins.

Picture it like this: two people read the same table aloud, one in English and one in Spanish. You don't speak Spanish. But you know they read in the same order, and you can hear when both say the same amount of money. That is enough to follow along. The script does exactly that.

The journey in one picture

Everything is lined up from big to small, in order



how the pieces connect · from two files to matched pairs

The steps go from big to small. The four matching steps all use the same two tools: the look-alike checker and the matchmaker.

Piece 1 · Reading the files

What it does: It opens the two files Azure produced from the PDFs and pulls out every table and every cell: the text, which row and column it sits in, which page it is on, and where on the page.

Why it is needed: The Azure files hold everything on every page. The script needs only the tables, in a tidy form it can work with.

How it solves it: It goes through each table and each cell, skips empty cells, and gives every cell a short name tag made of its page, its table and its place in the table. That tag is how the cell is known everywhere afterwards, including in the results.

In the code: `load_tables()`

Piece 2 · Gluing split tables back together

What it does: It finds tables that a page break cut in two and joins them back into one table.

Why it is needed: Spanish is longer, so a table that fits on one English page often runs onto the next Spanish page. Azure reports the two halves as two separate tables. If they stay separate, the English table has nothing whole to be compared with.

How it solves it: For each table it asks: is this just the rest of the table before it? It says yes when both have the same columns, the second one is on a following page, and either the header row is printed again at the top or the columns sit at the same places across the page. When it joins two halves, it counts the rows straight through, and it puts aside the repeated header, because that is only page decoration, not real content.

In the code: `stitch_tables()` and `_is_continuation()`

Piece 3 · The look-alike checker

What it does: It looks at one piece of English text and one piece of Spanish text and says how alike they look: not alike, a little alike, or very alike.

Why it is needed: To line things up, the script keeps asking "does this English thing go with that Spanish thing?" It cannot translate, so it needs a way to judge "these look like the same thing" without understanding the words.

How it solves it: It looks for clues that survive translation:

- **Money and percentages.** They stay the same in both languages. This is the strongest clue.
- **Identical words.** Codes, names and short forms are often the same on both sides.

- **Similar spelling.** "Specialist" and "especialista" share most of their letters. It notices this by comparing small chunks of letters.
- **Length.** A translation is a bit longer or shorter, but never wildly different.

It combines the clues into one "how alike" answer. If a clue is missing on both sides, for example neither text has any money in it, that clue is simply left out instead of counting against the pair.

In the code: `profile_similarity()` and `text_similarity()`

Piece 4 · The matchmaker

What it does: It takes two lists that are in the same order, for example the English rows and the Spanish rows, and pairs them from top to bottom without ever crossing over.

Why it is needed: Simple "first with first, second with second" matching falls apart as soon as one side has an extra item, because everything after it shifts. The matchmaker must be able to skip an item that has no partner, and to join two items on one side with one on the other when Azure split something.

How it solves it: Picture two queues of people standing in the same order. At each step the matchmaker can:

- pair the two people at the front,
- let one person through without a partner, or
- group a few people on one side with one person on the other.

Pairing look-alikes earns points. Leaving someone without a partner costs points. It tries every possible way through both queues and keeps the one with the best overall result. Because it never pairs the front of one queue with the back of the other, the order is always kept.

This one tool is reused for tables, for columns and for rows. Understand it once and much of the file makes sense.

It has two small helpers:

- **Same place, a little more trust.** Items at the same place in their lists, for example both near the top, get a small boost.
- **Clear winners stand out.** When one partner is clearly the best, close runners-up are pushed down so they don't get paired by mistake.

In the code: `align_sequences()`

Piece 5 · Matching whole tables

What it does: It decides which English table goes with which Spanish table, and which tables have no partner at all.

Why it is needed: There is no point comparing rows until the right tables sit side by side. The two PDFs can have a different number of tables, and some tables exist in one language only.

How it solves it: It compares every English table with every Spanish table, mostly by the header row, then by all the text, the shape (how many columns, how big) and where the table sits in the document. Then the matchmaker pairs them in order.

It is strict on purpose:

- A weak pair is never allowed.
- A table with no good partner is left alone rather than forced onto a neighbour.
- If gluing missed a split, it can still pair two pieces with one table, but only when each piece is a good match on its own.

A table left alone is not an error. For example, the Spanish document has some tables that the English one writes as normal paragraphs.

In the code: `align_logical_tables()`

Piece 6 · Matching columns

What it does: Inside a pair of tables, it decides which English column goes with which Spanish column.

Why it is needed: The two tables may not have the same columns. Spanish might have an extra column, or Azure might read the columns slightly differently. Rows can only be compared fairly once you know which column to compare with which.

How it solves it: It describes each column by its header and by the text below it, and compares the columns, giving a little extra trust to columns in the same position. The matchmaker pairs them in order, one column with one column. An extra column with no partner is left out.

The result is a **column map**: "this English column goes with that Spanish column". The next pieces use it.

In the code: `align_columns()`

Piece 7 · Matching rows

What it does: Inside a pair of tables, it decides which English row goes with which Spanish row.

Why it is needed: This is where the old logic failed. Rows don't line up one to one:

- Azure sometimes splits one row into two.
- A short leftover line can show up as a row of its own.
- One side can have a row the other doesn't.
- Headers get printed again after page breaks.

How it solves it: First it sets aside the repeated headers. They are page decoration, and they get paired separately later. Then, for each possible pair of rows, it asks the look-alike checker to compare them column by column, using the column map. While doing that it:

- **trusts long cells more than short ones**, because a short cell like "None" appears everywhere and proves little,
- **trusts a pair less if the two rows fill different numbers of columns**, because real partners usually have the same shape,
- **gives some trust to rows at the same place in their tables**, so rows with no shared words still land in the right spot,
- **keeps header rows with header rows.**

The matchmaker then pairs the rows in order. It may join a split row on one side with a single row on the other, but only when the joined version really is a better match than any simple one-to-one pair. Without that check it would join rows everywhere, because joined rows always look a bit more alike simply by having more text.

In the code: `align_rows()` and `row_block_score()`

Piece 8 · Matching cells

What it does: Inside each matched pair of rows, it pairs the actual cells and writes down each pair as one result.

Why it is needed: The final goal is cell to cell: this English cell is that Spanish cell. Tables, columns and rows are the route to get there.

How it solves it: For each English column, it takes the cells in the matching Spanish column, within the matched rows. Usually that is one cell on each side, so they pair directly. If one side has the text split over two cells, they are joined and paired with the single cell on the other side.

Each pair is recorded with how alike the two cells look, how they were matched, and where both cells sit. A pair that looks only a little alike is **flagged for a person to check**. A flag means "please check", not "wrong": correct pairs with no shared words often look only a little alike.

Headers printed again after page breaks are paired here too, by their order: the first repeat on one side with the first repeat on the other. If one side never repeats its header, its first header stands in as the partner.

In the code: `_pair_cell_lists()`, `map_repeated_headers()` and `map_table_pair()`

Piece 9 · A second chance for leftovers

What it does: It gives cells that are still unmatched one more chance to find a partner.

Why it is needed: Row matching works on whole rows and is careful, so now and then a cell gets left behind even though its partner is there.

How it solves it: For each leftover English cell, it looks only at leftover Spanish cells that are in the matching column and in roughly the same part of the table. It pairs the best look-alikes first, and each cell can be used only once. Anything still left after this goes on the list of unmatched cells for a person to review.

In the code: `rescue_unmatched_cells()`

Piece 10 · The result files

What it does: It writes the results to files.

Why it is needed: Different people need different things. Other code needs the matches in a fixed format, reviewers need a list to check, and a developer needs to see why a decision was made.

How it solves it: It writes four files:

File	What it holds	Who it is for
`table_cell_alignments.json`	Every matched pair	The main result, shaped like the pipeline's own table output
`table_mappings.csv`	One line per matched pair	Reviewing in Excel
`table_anomalies.csv`	Unmatched cells, and pairs flagged for checking	A to-do list for reviewers

File	What it holds	Who it is for
`table_segment_report.json`	Every decision: what was glued, which tables were paired, how rows were matched	Finding out why something went wrong

The script does not feed these results into the pipeline by itself. That connection still has to be built.

In the code: `write_outputs()`

Piece 11 · The picture previews

What it does: It draws the matches on the real PDF pages, English page next to Spanish page, so a person can check them by eye.

Why it is needed: Checking matches in a spreadsheet is slow. A picture shows a wrong match at a glance.

How it solves it: It turns the pages into images and draws a box around each matched cell: red on the English side, green on the Spanish side, blue around each table, and grey for cells with no partner. A matched pair carries the same tag on both sides, so you can spot the partner quickly.

Because the Spanish document is longer, it puts pages side by side based on the matches, not simply page by page. It saves one picture per page pair, plus one PDF with all of them. This step needs the two PDFs as well as the Azure files, and it can be switched off.

In the code: `save_table_alignment_previews()`

Helpers that support everything

The settings panel

What it is: A block at the top of the file that holds every setting affecting a decision: how much each clue counts, how good a match must be to be accepted, and how much it costs to leave something without a partner.

Why it exists: So all the tuning sits in one place instead of being scattered through the code.

Keep in mind: these settings were chosen by trial and error on one English–Spanish sample. Another kind of document or another language may need them adjusted. You don't need to understand each setting to understand the logic.

The optional meaning helper

What it is: An extra way to compare texts by their meaning, using Gemini through `pipeline.py`.

Why it exists: The look-alike checker relies on shared spelling and numbers. That works for English and Spanish, but not for languages with a different alphabet, such as Korean.

How it works: It is off unless you ask for it when you run the script. If it can't connect, the script carries on without it.

The control room

What it is: The main function, `run_table_segmentation()`, plus the command you type to run the script.

Why it exists: Something has to run all the pieces in the right order.

How it works: It finds the two Azure files (and the PDFs, if available), runs every piece in order, prints progress as it goes, and writes the results. When you feel lost in the file, start here: it reads like a table of contents.

The whole story in one paragraph

Tables in a translated PDF don't line up neatly with the original, because the translation is longer and page breaks fall in different places. This script fixes that in steps. It reads the tables from both Azure files and glues back together tables that page breaks cut in two. Then it lines everything up from big to small: tables, then columns, then rows, then cells. At every step it relies on one fact, that a translator never changes the order of a table, and on clues that survive translation, such as money amounts and similar spellings. It can skip things that exist on one side only and join things Azure split. Cells it can't place get a second chance, and anything still unmatched is listed for review. Finally it writes the results as files, and as side-by-side pictures for checking by eye.

What comes next

We go deeper one level at a time, and move on only when the current level makes sense.

1. Each piece again, with a small made-up example showing what goes in and what comes out.
2. The decisions inside each piece, still in words.
3. Where the settings come in, and what changing them would do.
4. The code itself, function by function.

Pick the piece you want to open first, or we go in order.

The 12,000-foot view: how each piece works

This level walks through each piece again, a little closer. For each piece you get what goes in, what happens step by step, what comes out, a small example, and anything good to know. There are still no numbers and no formulas.

Words used from here on

Word	What it means
Cell	One box of a table, with its text
Name tag	The short label each cell gets: its page, its table and its place in the table
Physical table	A table exactly as Azure reported it, even if a page break cut it in two
Logical table	One real table, after the cut halves are glued back together
Look-alike profile	The clues worked out once for each text (cleaned text, money amounts, words, letter chunks), so later comparisons are quick
Column map	The list saying which English column goes with which Spanish column
Row block	One or more English rows matched with one or more Spanish rows
Match record	One written-down pair: this English cell goes with that Spanish cell, plus how they were matched and how alike they look
Unmatched list	Cells that found no partner, kept for a person to review

Piece 1 at 12,000 feet · Reading the files

Goes in: one Azure file per language. Azure has already read the PDF. Its file lists the pages, and for each table its cells: the text, the row and column, whether the cell stretches over several rows or columns, whether it is a header, and the outline of the cell

on the page.

Steps:

1. Open the file and find the list of tables.
2. Name the tables in the order they appear in the document.
3. Work out which page each table is on, by checking which page its text belongs to. If that fails, use the page written on the table's outline.
4. Go through the cells. Skip any cell with no real text: empty, or holding only a checkbox mark that Azure adds.
5. Turn the cell's outline into a simple box (left, top, right, bottom), converted into the same units the PDF uses, so it lines up with the page later.
6. Note the page the cell itself sits on. Usually that is its table's page.
7. Give the cell its name tag: page, table, place in the table.
8. Work out the cell's look-alike profile once and keep it with the cell, so later comparisons don't redo the work.

Comes out: a list of physical tables, each holding its cells.

Example: the cell "Specialist visit" in the first table on page two gets a tag that says exactly that: page two, first table, and its place in that table.

Good to know:

- Different Azure versions spell some field names in two styles. The reader accepts both.
- The cell part of a tag can have gaps, because empty cells are skipped. That is normal.
- A table with no usable cells at all is dropped.

Piece 2 at 12,000 feet · Gluing split tables

Goes in: the physical tables of one document, in reading order.

Steps:

1. The first table starts the first logical table.
2. Each next table is compared only with the last piece of the current logical table, never with anything earlier.
3. It is glued on only if all of these are true:
    - it has the same number of columns,
    - it is on a later page, not the same page,
    - it is only a page or so later, not far away,

    - and either its header row repeats the previous header, or, when there is no repeated header, its columns sit at the same places across the page.
4. If any check fails, the table starts a new logical table.
5. Once all tables are grouped, the rows inside each logical table are counted again straight through all its pieces, so the second piece carries on where the first piece ended.
6. The header rows at the top of every piece after the first are marked as **repeated headers**. They are kept, but set aside from row matching.
7. Every decision is written down with its reason, for the report.

Comes out: a list of logical tables, plus a log of every glue decision.

Example: Spanish page one has a table with a header and the primary care row. Spanish page two has a table with the same header again, then the specialist row and the drugs row. They have the same columns, the second is on the next page, and the header repeats, so they become one logical table: header, primary care, specialist, drugs. The second header is set aside.

Good to know:

- A short paragraph between the two halves does not stop the gluing, as long as the checks pass.
- If gluing misses a split, Piece 5 has a backup: it can still pair two pieces with one table.
- Logical table names start with a letter for the side: one letter for the original, another for the translation.

Piece 3 at 12,000 feet · The look-alike checker

Goes in: two pieces of text, one English and one Spanish.

Steps:

1. **Clean both texts.** Remove checkbox marks, remove accents (é becomes e), make everything lowercase, and squeeze extra spaces.
2. **Collect the money amounts and percentages.** A price written with a dot and the same price written with a comma, the Spanish way, count as the same amount. Percentages are kept apart from plain amounts, so a percentage never matches a price that happens to use the same digits.
3. **Collect the words.** Very short words like "a", "to", "de" and "al" are ignored, because they appear everywhere and prove nothing.
4. **Cut each text into small chunks of letters** and see how many chunks the two texts share. This is what catches words spelled alike in both languages.

5. **Compare the lengths.** A translation is a bit longer or shorter, never wildly different.
6. **Combine the clues** into one answer, from "nothing alike" to "identical". Money counts most. A clue missing on both sides, for example no money in either text, is left out rather than counted against the pair.

Comes out: one "how alike" answer.

Example:

- "$25 copay/visit" and "Copago de $25/visita": the same amount appears on both sides, "copay" and "copago" share letter chunks, and so do "visit" and "visita". Result: quite alike.
- "Common Medical Event" and "Copago de $25/visita": no shared amount, almost no shared chunks. Result: not alike.

Good to know:

- It works because English and Spanish share an alphabet.
- Two empty texts count as identical. One empty and one not count as nothing alike.
- A correct pair can look only a little alike when the words are completely different, such as "Not covered" and "Sin cobertura". The other pieces make up for that using order and position.

Piece 4 at 12,000 feet · The matchmaker

Goes in: two lists in the same order (tables, columns or rows), a way to judge how alike any proposed pair is, and a few settings: what it costs to leave an item alone, how weak a pair may be before it is refused, and how many items may be grouped together.

Steps:

1. Start at the front of both lists with an empty running score.
2. At every point, look at each allowed move:
   - pair the front English item with the front Spanish item,
   - leave the front English item alone,
   - leave the front Spanish item alone,
   - group a few items on one side with a single item on the other.
3. Each move changes the running score. A pair adds how alike the two sides look; a group counts for all the items it covers, minus a small charge for grouping. Leaving an item alone takes away a fixed cost. A pair that is too weak is not allowed at all.
4. For every point in the two lists, remember only the best way found to get there. Comparisons already made are remembered too, so nothing is checked twice.
5. When both lists are used up, walk back along the best route and read off the decisions.

Comes out: a list of **blocks**. Each block says either "these English items go with these Spanish items" (one with one, one with several, or several with one), or "this item has no partner".

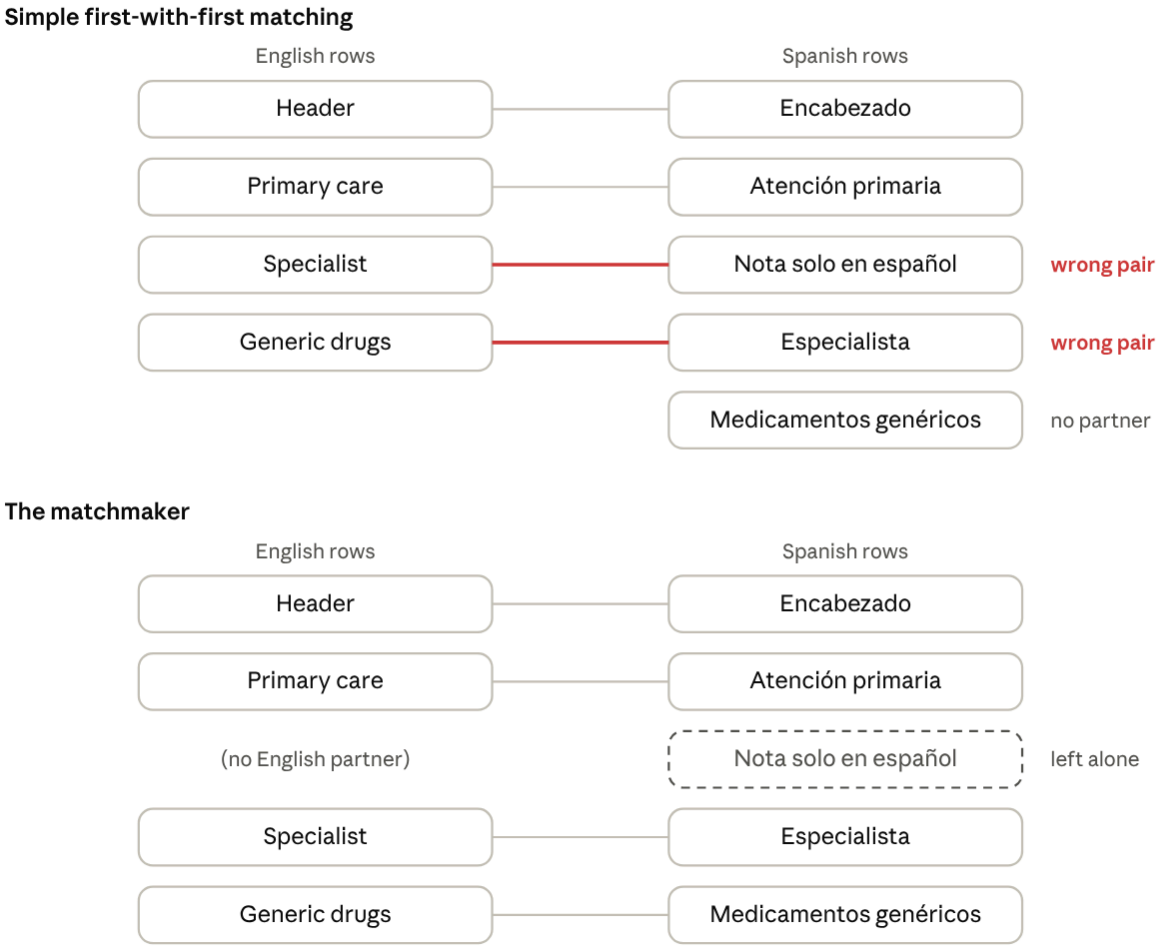
Example: English rows are header, primary care, specialist, drugs. Spanish rows are header, primary care, a note that exists only in Spanish, specialist, drugs. The matchmaker pairs header with header and primary care with primary care, leaves the Spanish note alone, then pairs specialist with specialist and drugs with drugs. Simple first-with-first matching would have paired specialist with the note, and every row after it would have been wrong. The drawing below shows this.

Two helpers:

- **Position hint.** It looks at where an item sits in its list (near the top, in the middle, near the bottom), not its exact place. Items at a similar spot get a little extra trust.
- **Clear-winner rule.** Used for tables. Each pair is compared with the best other option either table has. A pair that is not the best choice for either side loses value, so a close runner-up does not get paired by mistake.

Good to know: it never groups several items with several items at once. This kind of method is called dynamic programming; tools that show the difference between two files work the same way.

The matchmaker leaves the extra row alone instead of shifting everything



first-with-first matching versus the matchmaker · one extra Spanish row

With simple first-with-first matching, one extra Spanish row makes every pair after it wrong. The matchmaker leaves that row alone, and everything after it stays correct.

Piece 5 at 12,000 feet · Matching whole tables

Goes in: the logical tables from both documents.

Steps:

1. For every possible English–Spanish table combination, build a score from four parts:
   - how alike the header rows are (this counts most),
   - how alike all the text in the two tables is,
   - how similar their shape is: the number of columns and the overall size,
   - whether they sit at a similar place in their documents.
2. Put all these scores in a grid, English tables down the side and Spanish tables across the top. The grid is saved for the report.

3. Apply the clear-winner rule from Piece 4, so close runners-up lose value.
4. Run the matchmaker with strict settings. A weak pair is refused, and leaving a table alone is an acceptable outcome.
5. Grouping is allowed only as two tables with one, and only when each of the two is a good match on its own. This is the backup for a split that gluing missed.
6. When two tables were grouped, they are joined into one temporary table. The join uses copies, so the original tables are not changed.
7. Every table left without a partner sends all its cells straight to the unmatched list.

Comes out: a list of table pairs to pass on to the next pieces, plus the tables left alone.

Example: English has a benefits table and a drug table. Spanish has a benefits table, an extra table that exists only in Spanish, and a drug table. Result: benefits with benefits, the extra Spanish table alone, drugs with drugs.

Good to know: when two tables get paired wrongly, the score grid in the report is the first thing to check. It shows what the right table scored compared with the one that was chosen.

Piece 6 at 12,000 feet · Matching columns

Goes in: one pair of tables, English and Spanish.

Steps:

1. For each column, gather two things: the text of its header cells, and, separately, the text of all its other cells.
2. For every possible English–Spanish column combination, judge how alike they are:
   - compare the headers (this counts most),
   - compare the text below the headers,
   - give a little extra trust to columns in the same position.
3. Run the matchmaker over the columns, in order. Columns are paired one with one only; columns are never grouped.

Comes out: the **column map**, a list saying which English column goes with which Spanish column. A column with no partner is simply not in the map.

Example: English columns: the event, the service, the cost. Spanish columns: the event, the service, the cost, and an extra column of notes. Result: event with event, service with service, cost with cost. The notes column has no partner.

Good to know: cells in a column that has no partner are never compared during row matching. That is one of the ways a cell ends up on the unmatched list.

Piece 7 at 12,000 feet · Matching rows

Goes in: one pair of tables, plus the column map from Piece 6.

Steps:

1. **Lay out each table by rows.** Each row holds its cells, sorted by column. Repeated headers are left out here.
2. **Learn what a normal row looks like** in this table, meaning how many columns a typical row fills. This is used to spot leftover fragments later.
3. **Judge a proposed pairing of rows.** The proposal can be one row with one row, or a few joined rows with one row. To judge it:
   - go through the columns using the column map, and ask the look-alike checker to compare the text in each column pair,
   - let long cells count more than short ones, because a short cell like "None" appears in many rows and proves little,
   - reduce the score a little when the two sides fill different numbers of columns, because real partners usually have the same shape,
   - add the position hint, so rows near the same spot get some trust,
   - give a small bonus when both rows are headers, and a small penalty when only one of them is.
4. **Guard against joining too much.** A joined pairing is allowed only if it does at least as well as the best simple one-with-one pair among those same rows. There is a little extra room when one of the joined rows is a fragment: a very short row that fills only a few columns, usually a leftover from a split.
5. **Run the matchmaker** over the rows, in order, using these judgements.

Comes out: a list of **row blocks**: which English row or rows go with which Spanish row or rows, and which rows have no partner.

Example: Azure split the English row "Tier 2 – Your Midrange-Cost Option" into two rows, "Tier 2 – Your" and "Midrange-Cost Option". The Spanish has it as one row. The two English rows joined match the Spanish row better than either half alone, so they are joined into one block.

Good to know:

- Every judgement is remembered, so the same pairing is never judged twice.
- Real partners often look only a little alike. What matters is that they look more alike than the other options do.

Piece 8 at 12,000 feet · Matching cells

Goes in: the row blocks from Piece 7, the column map from Piece 6, and the two tables.

Steps:

1. **Repeated headers first.** List the repeated headers on each side in the order they appear. Pair the first with the first, the second with the second, and spread them evenly if one side has more. If one side has no repeated headers, its first header stands in as the partner. Inside a header, each cell pairs with the cell in the same row and the matching column. If no cell sits there, it takes the most look-alike cell in that column.
2. **Then the row blocks.** For each block that has rows on both sides, go through the English columns. For each one, find its Spanish column in the column map and gather the cells on both sides.
3. **Pair the gathered cells:**
   - one cell on each side: pair them,
   - one on one side and several on the other: join the several into one text and compare it with the single cell,
   - several on both sides, which is rare: let the matchmaker pair them.
4. **Write a match record** for every English cell. When several English cells go with one Spanish cell, each of them gets its own record pointing at the same Spanish cell, so no cell disappears.
5. **Fill in the record:** both name tags, both texts, where both cells sit on their pages, how alike they look, what kind of match it is, which step made it, and any flags.
6. **Add flags.** A pair that looks only a little alike is flagged as low confidence. Any match that is not one with one is flagged with its kind as well.

Comes out: the match records for this table pair.

Example:

- "$25 copay/visit" and "Copago de $25/visita": one record, one with one, no flag.
- "Visit to a provider office" and "Visita al consultorio de un proveedor": one record, one with one, flagged as low confidence, even though the pair is correct.

Good to know: each record holds the fields the pipeline already understands, plus the extra fields for checking. The flags are there to point a reviewer at pairs worth a second look, not to mark them as wrong.

Piece 9 at 12,000 feet · A second chance for leftovers

Goes in: the cells in one table pair that still have no partner after Piece 8, on both sides.

Steps:

1. Take each leftover English cell and look at each leftover Spanish cell.
2. Skip the Spanish cell if it is not in the matching column. (If the English column has no partner in the column map, any column is allowed.)
3. Skip it if the two cells sit too far apart in the table, judged by how far down the table each one is.
4. Ask the look-alike checker how alike the two cells are. Keep the pair as a candidate only if they look alike enough.
5. Sort all the candidates, best first.
6. Go down the sorted list and accept each pair, as long as neither cell has already been used.

Comes out: extra match records, each marked as made by the rescue step. Cells still left after this go on the unmatched list.

Example: a Spanish cell was left out because Azure split its row differently from the English one. The rescue step finds its English partner in the same column, in the same part of the table, and pairs them.

Good to know:

- It takes the best pair first and never goes back to rethink. That is simple and fast, and fine for the few cells that reach this step.
- It only looks inside one table pair, never across different tables.

Piece 10 at 12,000 feet · The result files

Goes in: all the match records, the unmatched lists, and every decision made along the way.

Steps:

1. **Group the records by English table.** The layout copies what the pipeline's own table matching produces: the page, the table, its partner tables, the outline of both tables, and the list of cell pairs sorted by row and column. That is what would let the pipeline use these results later.
2. **Work out summary figures:** how many cells each side has, how many were matched, what share was matched, how alike the pairs look on average, and how many were flagged.
3. **Write the main result file** (`table_cell_alignments.json`): the summary plus the grouped matches.

4. **Write the report** (`table_segment_report.json`): every glue decision with its reason, the table score grid, each table pair's column map and row blocks, and every unmatched cell.
5. **Write the pairs spreadsheet** (`table_mappings.csv`). Its first columns copy the layout of the pipeline's existing mappings file, with the cell's name tag where the pipeline puts its paragraph tag. Extra columns hold the table details, plus a check of whether the money amounts agree on both sides.
6. **Write the problems spreadsheet** (`table_anomalies.csv`): unmatched English cells, unmatched Spanish cells, and pairs flagged as low confidence.

Comes out: four files in the output folder.

Good to know: a cell lands on the unmatched list in one of three ways:

- its whole table had no partner (Piece 5),
- its column had no partner (Piece 6),
- or it was still alone after the second chance (Piece 9).

The report holds what you need to tell which: tables left alone, and each table pair's column map.

Piece 11 at 12,000 feet · The picture previews

Goes in: the results, plus the two PDFs.

Steps:

1. **Decide which pages to show together.** From the matches, list which English page goes with which Spanish page. One English page can go with several Spanish pages, because the Spanish document is longer.
2. **Decide what to draw on each page:**
   - a box around each matched cell, labelled with its tag,
   - a grey box, marked unmapped, around each cell on that page with no partner,
   - an outline around each table, built from its matched cells.
3. **Turn each page into an image.**
4. **Draw the boxes.** Tables first, then cells on top. Red on the English side, green on the Spanish side, blue for tables, grey for unmatched. Each cell's label sits inside its top-left corner on a see-through tag, so the text underneath stays readable. Table labels sit just above the table.
5. **Label the Spanish cells with their English partner's tag.** That way the same tag appears on both sides of a matched pair. A setting can show both tags instead.

6. **Put the two images side by side** and save the picture. Then gather all the pictures into one PDF, shrinking very large images so the file stays a manageable size.

Comes out: one picture per page pair, and one PDF holding all of them.

Good to know:

- If anything fails here, the rest of the run is not affected. The result files are already written.
- It tries a few common fonts for the labels and falls back to a basic one if none is found.
- It can be switched off, and it is skipped when the PDFs are not available.

Helpers at 12,000 feet

The settings panel

The settings are grouped by piece: one group for the look-alike checker, then one each for gluing, tables, columns, rows, and cells with rescue. Every group has the same three kinds of setting:

- **How much each clue counts**, for example money against spelling, or header against body text.
- **How good a match must be** before it is accepted.
- **How much it costs to leave something alone.**

What turning them does, in plain terms:

- Make leaving things alone **more costly**, and the script pairs more eagerly. More cells get matched, but more of them wrongly.
- Ask for **better matches** before accepting, and the script gets stricter. Fewer wrong pairs, but more cells end up unmatched.
- Let a clue **count more**, and the script leans on that clue more when deciding.

The optional meaning helper

1. When you turn it on, it collects the text of every cell in both documents.
2. It loads `pipeline.py` and asks it for a "meaning fingerprint" of each text, using Gemini.
3. From then on, whenever two cells are compared, the answer mixes "do they mean the same thing" with the look-alike checker's answer.
4. If anything goes wrong, such as missing login details or no connection, it prints a short message and the script carries on with the look-alike checker alone.

The control room

`run_table_segmentation()` runs everything in this order:

1. Read both Azure files (Piece 1).
2. Glue split tables on both sides (Piece 2).
3. Turn on the meaning helper, if asked.
4. Match whole tables (Piece 5).
5. For each table pair: columns, rows, cells, second chance (Pieces 6 to 9).
6. Work out the summary and write the result files (Piece 10).
7. Draw the previews, if the PDFs are available (Piece 11).

It prints a progress line after each stage.

The command you type gives it its inputs in one of two ways:

- **Point it at a folder** that holds both Azure files. It works out which file is English from the folder's name, and finds the PDFs from the state file the pipeline saved there.
- **Name the two Azure files directly**, and optionally the two PDFs.

There are also options to skip the previews, print a few sample matches on screen, turn on the meaning helper, and run quietly.

Next level: the 10,000-foot view

The next layer takes the same pieces one step closer:

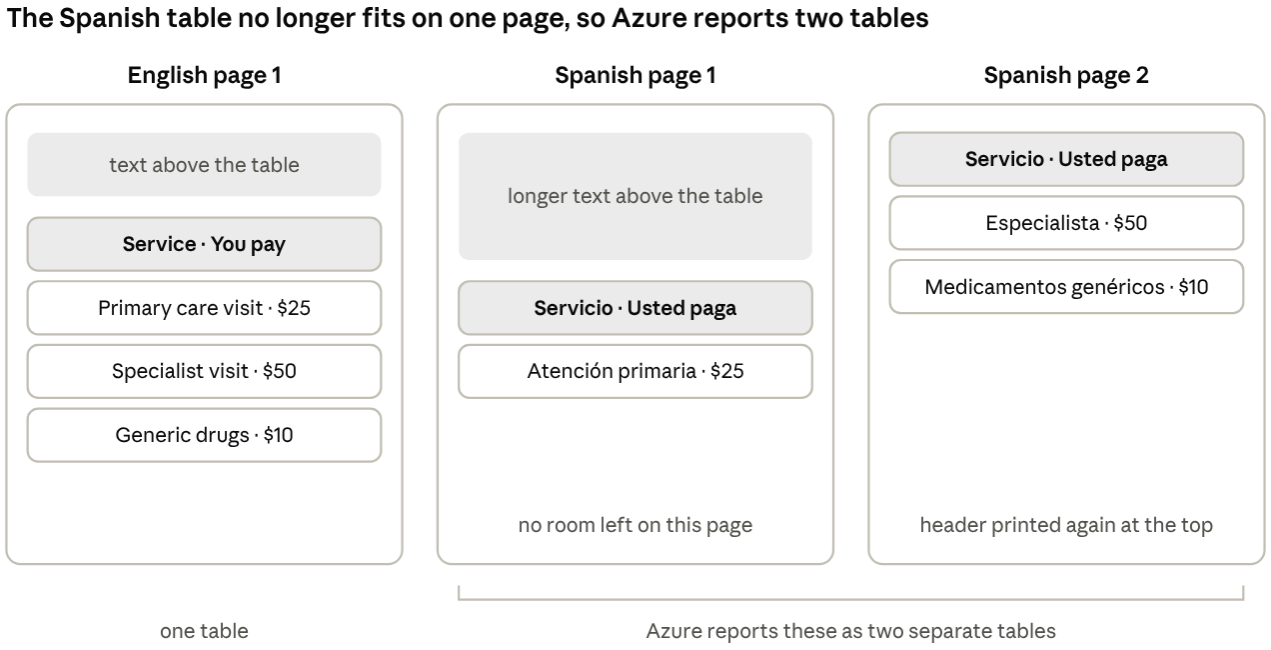
- the exact questions each piece asks, in the order it asks them,
- what the script does in the awkward cases, such as a cell that stretches over several rows, or a table with no header,
- one small example table followed all the way through every piece.

Still in plain words. The settings and their values come in only at a later level, when the logic is already clear.

The 10,000-foot view: pictures and one running example

This level shows each piece as a picture with a small example. Each section answers three things: **the problem in one sentence**, **the fix**, and **the question the piece keeps asking**. There are still no scores and no settings.

The running example



the running example · one English table, two Spanish pieces

One small, made-up example runs through this whole level. English has one table: a header and three rows. In Spanish, the text above the table is longer, so the table no longer fits on one page. Azure reports it as two separate tables, one on each page, with the header printed again on the second page.

Cell tags read like this: **p1t1c4** means page 1, table 1, cell 4. Each document has its own tags, so an English cell and a Spanish cell can carry the same tag.

Piece 1 at 10,000 feet · Reading the files

The problem in one sentence: Azure's file describes everything on every page in its own format, but the script needs only the table cells, in one tidy form.

The fix: copy out each table cell, keep only what later pieces need, skip empty cells, and give every cell a name tag.

> The question it keeps asking: "Does this cell have real text? If so, what is its tag, where is it, and what does it look like?"



one cell, from Azure's file to the script's tidy form

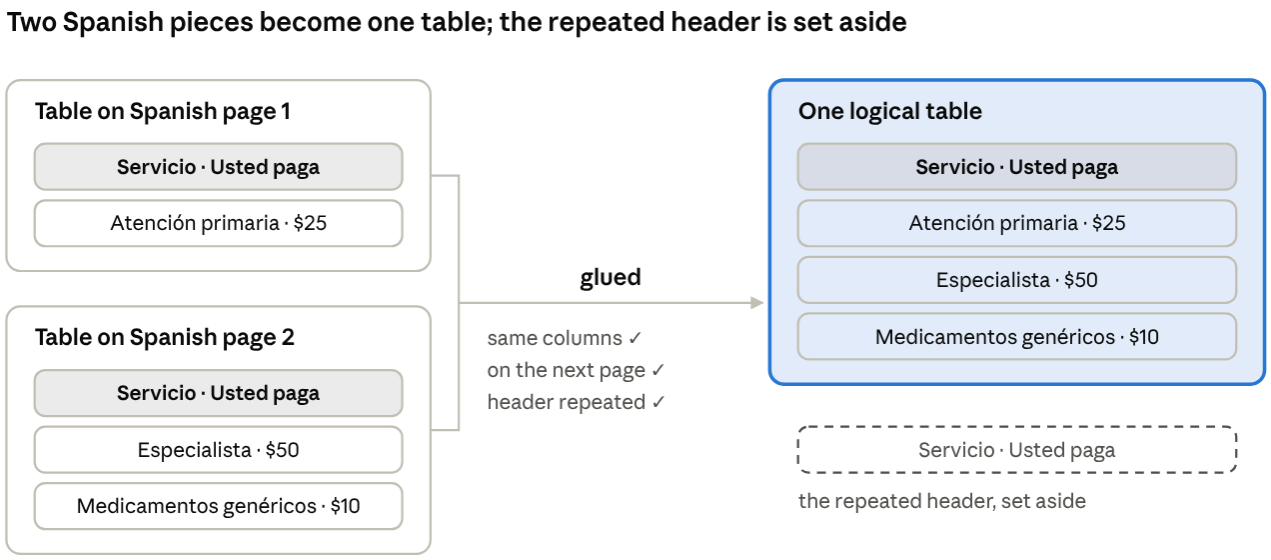
In the example: each row has two cells, the service and the amount. The English cell "$25 copay" becomes **p1t1c4**, with its page, its place in the table, its box on the page, and its look-alike profile ready for later. The Spanish file is read the same way, on its own: "Copago de $25" on Spanish page 1 also becomes p1t1c4, in the Spanish document.

Piece 2 at 10,000 feet · Gluing split tables

The problem in one sentence: a page break cut the Spanish table in two, so there is no whole Spanish table to compare with the English one.

The fix: glue a table onto the one before it when it looks like a continuation, count the rows straight through, and set the repeated header aside.

> The question it keeps asking: "Is this table just the rest of the one before it?"



gluing · two Spanish pieces become one logical table

The checks, in order. Any "no" starts a new table instead.

1. Same number of columns as the table before?
2. On a later page?
3. Only a page or so later?
4. Header repeated, or columns sitting at the same places across the page?

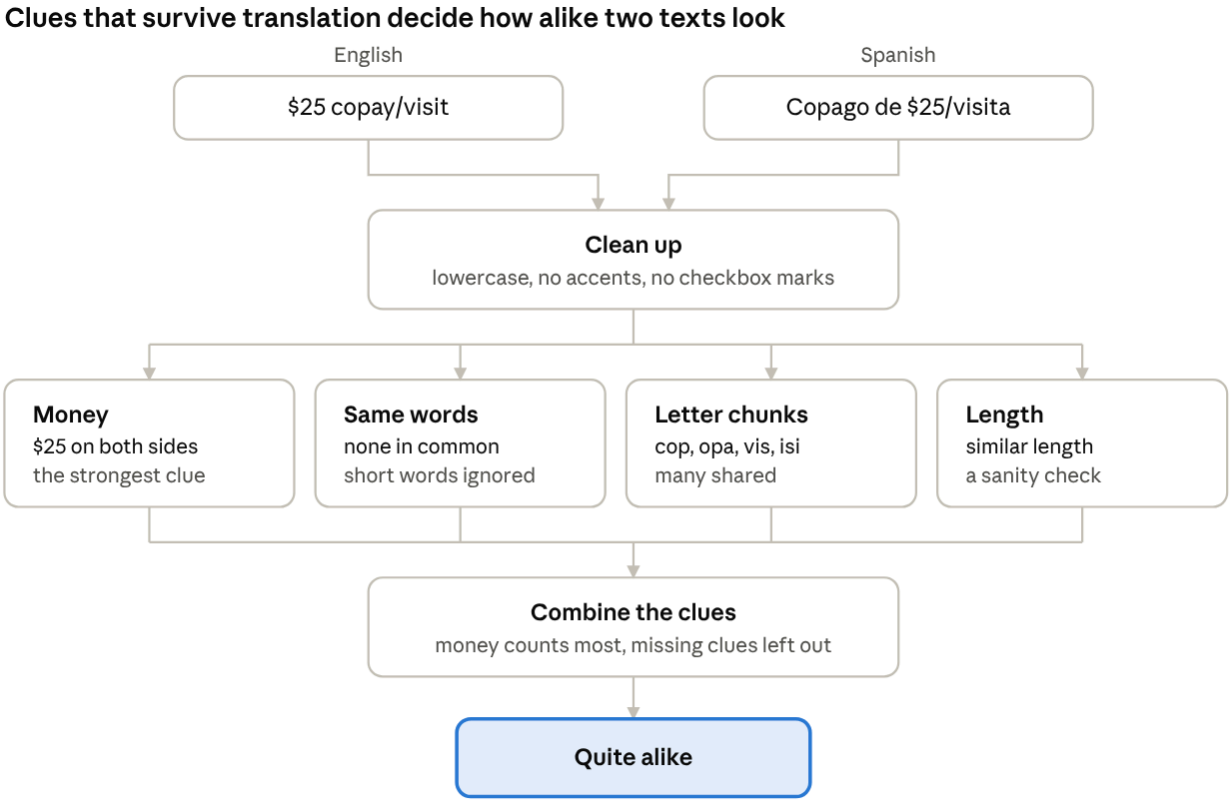
In the example: both Spanish pieces have two columns, the second is on the next page, and its header repeats, so they are glued. The Spanish rows now read: header, Atención primaria, Especialista, Medicamentos genéricos. That is the same order as the English rows, which is what every later piece relies on.

Piece 3 at 10,000 feet · The look-alike checker

The problem in one sentence: the script must judge whether an English text and a Spanish text say the same thing, without being able to translate.

The fix: look for clues that survive translation, and combine them into one answer.

> The question it keeps asking: "How alike do these two texts look?"



the look-alike checker · one English text against one Spanish text

In the example: three comparisons the later pieces will ask for.

English	Spanish	Answer	Why
$25 copay	Copago de $25	quite alike	same amount, similar spelling
Specialist visit	Especialista	a little alike	similar spelling only
You pay	Copago de $50	not alike	nothing in common

The second row is a correct pair that looks only "a little alike". The checker alone can't be sure about it. The order of the rows and their position make up for that later.

Piece 4 at 10,000 feet · The matchmaker

The problem in one sentence: pairing first with first breaks as soon as one side has an extra item.

The fix: walk both lists in order. At each step, pair, skip or group, and keep the route with the best overall result.

> The question it keeps asking: "What is the best way to pair these two lists without changing their order?"



the matchmaker's four moves

A trace. This is the winning route for the extra-row example from the 12,000-foot view, where Spanish has a note that English doesn't. The matchmaker weighs every possible route; this table shows the one that wins.

Step	Front of the English list	Front of the Spanish list	Move chosen
1	Header	Encabezado	Pair
2	Primary care	Atención primaria	Pair
3	Specialist	Nota solo en español	Skip Spanish
4	Specialist	Especialista	Pair
5	Generic drugs	Medicamentos genéricos	Pair

At step 3, pairing "Specialist" with the note would earn very little, and every pair after it would go wrong too. Paying the small cost to skip the note gives a much better total, so that route wins.

In the running example: after gluing, both lists hold the same rows in the same order, so every step is a plain pair.

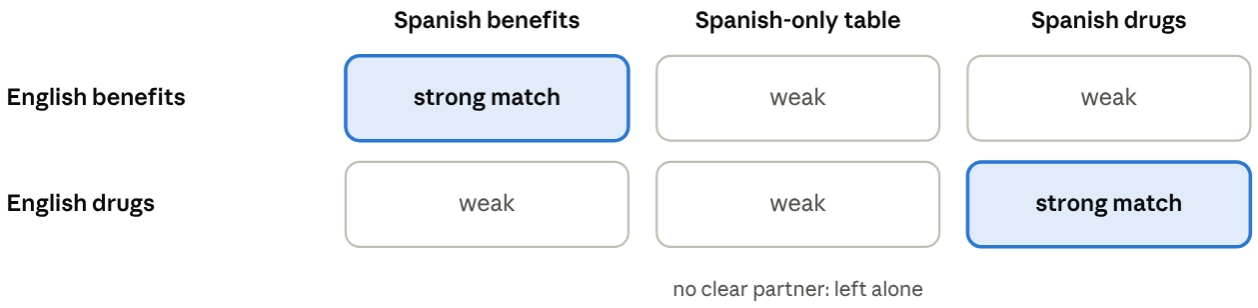
Piece 5 at 10,000 feet · Matching whole tables

The problem in one sentence: the two documents can have a different number of tables, and some tables exist in one language only.

The fix: score every English table against every Spanish table, keep only the clear winners, and let the matchmaker pair them in order.

> The question it keeps asking: "Which Spanish table is clearly the best partner for this English table, if any?"

Each English table goes to its clearly best Spanish partner



Each square compares two tables by their header row, all their text, their shape and their place in the document.

the table score grid · strong squares are the pairs

A bigger example. The running example has one table on each side after gluing, so it pairs straight away. The grid above imagines a larger document: English has a benefits table and a drug table, and Spanish has those two plus a table that exists only in Spanish.

Each English table has one clearly strong square, and those become the pairs. The Spanish-only table has no strong square in its column, so it is left alone and its cells go to the unmatched list. That is the right outcome: the English document covers that content as normal paragraphs.

Why "clearly": if a table had two squares that were almost equally good, the clear-winner rule would weaken both. A close runner-up never gets paired by mistake.

Piece 6 at 10,000 feet · Matching columns

The problem in one sentence: rows can only be compared fairly once we know which column goes with which.

The fix: describe each column by its header and its contents, and pair the columns in order, one with one.

> The question it keeps asking: "Which Spanish column holds the same kind of information as this English column?"

the column map · which English column goes with which Spanish column

In the example: Service goes with Servicio, and You pay goes with Usted paga. "Service" and "Servicio" look alike. "You pay" and "Usted paga" don't, but the amounts below them match strongly, and the columns sit in the same positions, so they still pair.

The drawing also imagines an extra Spanish "Notas" column. It has no English partner, so it is left out of the column map, and its cells are not compared in row matching. That is one way a cell ends up on the unmatched list.

Piece 7 at 10,000 feet · Matching rows

The problem in one sentence: rows don't always line up one to one, because Azure splits rows, leftover lines appear, and one side can have an extra row.

The fix: judge every possible row pairing cell by cell through the column map, adjust for length, shape, position and headers, then let the matchmaker pair the rows. A split row is joined only when that clearly helps.

> The question it keeps asking: "Which Spanish row, or rows, hold the same content as this English row?"

How two rows are judged

judging one row pairing · cell by cell, then four adjustments

In the example: "Specialist visit · $50 copay" against "Especialista · Copago de $50". The service cells look a little alike, because of similar spelling. The amount cells look quite alike, because the amount is the same. Both rows fill two columns, both sit in the middle of their tables, and neither is a header. Result: a good match.

The same English row against the Spanish primary care row would do worse: the amounts differ, and the position is off by one row. The matchmaker uses these judgements to pick the best route.

When a row was split

a split row · two English halves joined to one Spanish row

Azure sometimes splits one row into two. Here the English row "Tier 2 – Your Midrange-Cost Option" came out as two rows. The script tries each half alone and both halves joined, and joins them only because the joined version matches better than either half. This example is for illustration; the running example has no split rows.

Piece 8 at 10,000 feet · Matching cells

The problem in one sentence: matched rows still hold several cells, and we need the exact cell-to-cell pairs.

The fix: inside each matched row block, pair the cells column by column through the column map, and write a record for every English cell.

> The question it keeps asking: "Within these matched rows, which Spanish cell goes with this English cell?"

three ways cells pair · and the records they produce

In the example: the record for the amount in the primary care row looks like this.

Field	Value
English cell	p1t1c4 · "$25 copay"
Spanish cell	p1t1c4 · "Copago de $25"
Kind of match	one with one
Made by	row matching
How alike	quite alike
Flag	none

The service cell in the same row, "Primary care visit" against "Atención primaria", gets its own record. It shares only a few letter chunks, so it carries a low-confidence flag even though the pair is right.

Piece 9 at 10,000 feet · A second chance for leftovers

The problem in one sentence: careful row matching sometimes leaves a cell behind even though its partner is right there.

The fix: give leftovers one more chance. Compare them only with nearby leftovers in the same column, and take the best pairs first.

> The question it keeps asking: "Is there a leftover Spanish cell nearby, in the same column, that looks enough like this one?"

the second chance · three checks, then best pairs first

In the example: nothing is left over, so this step has nothing to do. It matters on messy real documents. A typical case is a cell whose row Azure split differently on the two sides, so row matching paired the rows but not that one cell.

Piece 10 at 10,000 feet · The result files

The problem in one sentence: different people need different things from the results.

The fix: write four files, each shaped for its reader.

> The question it answers: "Who needs what, and in which shape?"

what goes into each result file

How to read the picture: match records feed three files. Flagged pairs go to the problems spreadsheet as well as the others. Unmatched cells go to the problems spreadsheet and the report. Decisions, such as what was glued and why, go only to the report.

In the example: `table_mappings.csv` gets one line per matched pair, for example English p1t1c4 "$25 copay" with Spanish p1t1c4 "Copago de $25". `table_anomalies.csv` lists only the flagged pairs, such as "Primary care visit" with "Atención primaria", because nothing was left unmatched.

Piece 11 at 10,000 feet · The picture previews

The problem in one sentence: checking matches in a spreadsheet is slow, and wrong matches are easy to miss.

The fix: draw the matches on the real pages, English and Spanish side by side, with the same tag on both cells of a pair.

> The question it keeps asking: "Which pages belong together, and where is each matched cell on them?"

a preview page pair · a sketch, not a real preview

In the example: the English table is all on page 1, but the Spanish table runs over pages 1 and 2. So there are two page pairs: English page 1 with Spanish page 1 (the primary care row), and English page 1 with Spanish page 2 (the specialist and drugs rows). Each pair becomes one picture, and both pictures go into one PDF.

The grey dashed box shows what an unmatched cell would look like. The running example has none.

The control room at 10,000 feet

The problem in one sentence: something has to run the pieces in the right order and hand each one what the previous one produced.

The fix: one main function calls each piece in turn and passes the results along.

the control room · who calls whom, in order

Solid arrows are calls; dashed arrows are what comes back. The names in the picture map to the code like this:

In the picture	In the code
Control room	`run_table_segmentation()`
Reader	`load_tables()`
Gluer	`stitch_tables()`
Table matcher	`align_logical_tables()`
Pair worker	`map_table_pair()`, which calls `align_columns()`, `align_rows()`, `map_repeated_headers()`, `_pair_cell_lists()` and `rescue_unmatched_cells()`
File writer	`write_outputs()`

In the picture	In the code
Preview maker	`save_table_alignment_previews()`

The settings panel is read by every piece. The optional meaning helper, when it is on, sits inside the look-alike checker.

Next level: the 7,000-foot view

The next layer opens the awkward cases each piece has to handle:

- a cell that stretches over several rows or columns,
- a table with no header row,
- a row with empty cells,
- two tables on the same page that look like one.

It also starts naming the settings each piece reads, still in words first, so you can see which knob changes which decision before any values appear.